

Full Sharded Data Parallel

Lorem Ipsum

Utshav Saha

sahaushno@gmail.com

11. Sharding: split one big thing into pieces

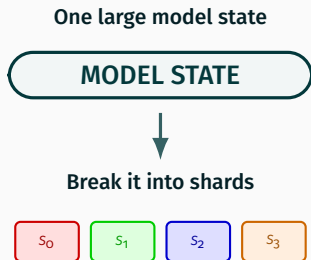
One large model state

MODEL STATE

Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

11. Sharding: split one big thing into pieces

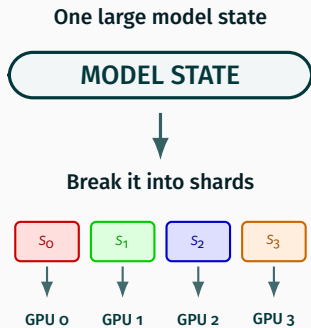


Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

- Split one large state into pieces.

11. Sharding: split one big thing into pieces

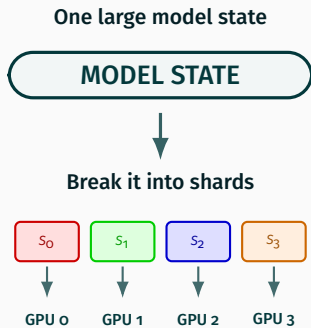


Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

- Split one large state into pieces.
- Give different pieces to different GPUs.

11. Sharding: split one big thing into pieces



Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

- Split one large state into pieces.
- Give different pieces to different GPUs.
- Reconstruct a full piece of the model only when computation needs it.

FSDP starts by **splitting** model state instead of **replicating** it.

12. What exactly is sharded?

DDP: replicated

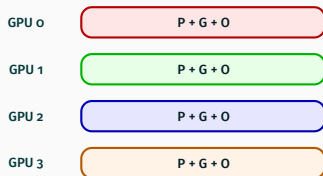


Every GPU keeps a full copy.

FSDP: sharded

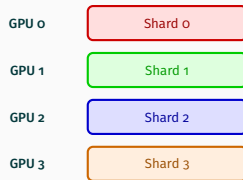
12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded



Each GPU keeps only its fraction.

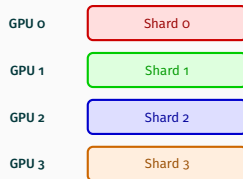
12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded



Each GPU keeps only its fraction.

P = parameters (weights), **G = gradients**, **O = optimizer states** (for Adam, moment estimates). FSDP can shard all three persistent model states.

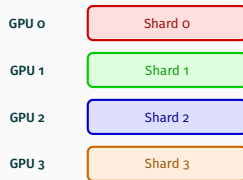
12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded



Each GPU keeps only its fraction.

P = parameters (weights), **G = gradients**, **O = optimizer states** (for Adam, moment estimates). FSDP can shard all three persistent model states.

Idealized persistent model-state memory per GPU scales roughly as $\frac{P+G+O}{N}$.

13. The first problem: a layer cannot compute from one shard

GPU 0 owns only



one piece of a layer's weight tensor

13. The first problem: a layer cannot compute from one shard

GPU 0 owns only

w_0

one piece of a layer's weight tensor

Layer operation
 $y = Wx$

?

The tension

Sharding saves memory, but the current layer normally needs its full parameter tensor to run the matrix operations.

13. The first problem: a layer cannot compute from one shard

GPU 0 owns only



one piece of a layer's weight tensor

Layer operation
 $y = Wx$

?

W means the **complete** tensor,
not only w_0 .

The tension

Sharding saves memory, but the current layer normally needs its full parameter tensor to run the matrix operations.



13. The first problem: a layer cannot compute from one shard

GPU 0 owns only



one piece of a layer's weight tensor

Layer operation
 $y = Wx$

?

W means the **complete** tensor,
not only w_0 .

The tension

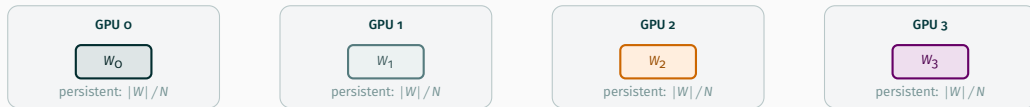
Sharding saves memory, but the current layer normally needs its full parameter tensor to run the matrix operations.



So FSDP must **temporarily collect** the missing parameter shards before computation.

14. All-Gather: temporarily rebuild the current FSDP unit

(a) Before. Each rank persists one disjoint shard W_i of this unit's parameter tensor.



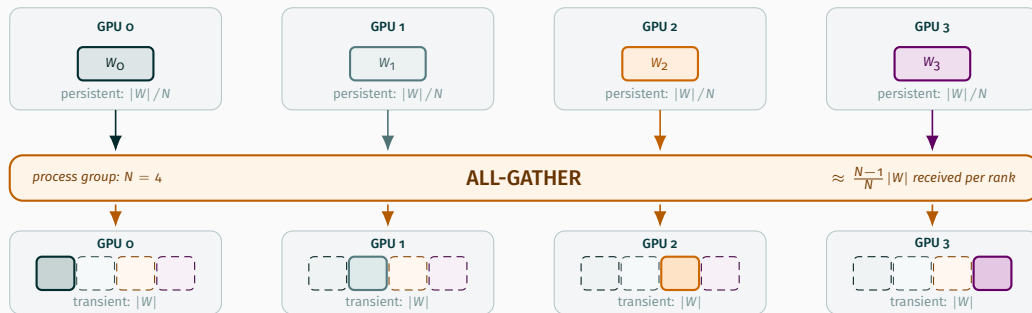
14. All-Gather: temporarily rebuild the current FSDP unit

(a) Before. Each rank persists one disjoint shard W_i of this unit's parameter tensor.



14. All-Gather: temporarily rebuild the current FSDP unit

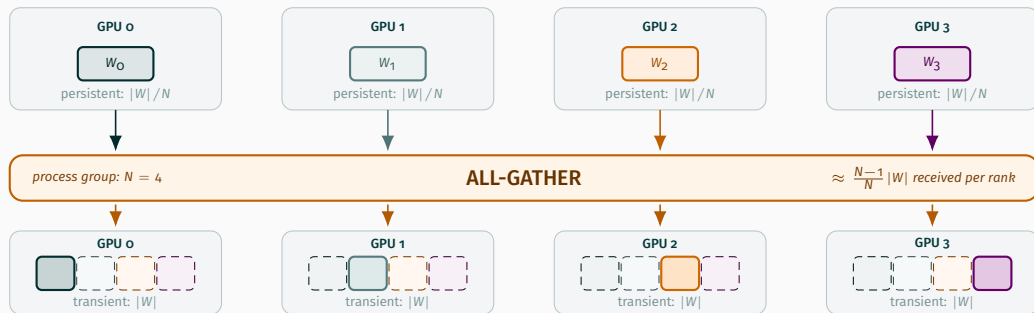
(a) Before. Each rank persists one disjoint shard W_i of this unit's parameter tensor.



(b) After. Every rank materializes $W = [W_0 \parallel W_1 \parallel W_2 \parallel W_3]$; solid = owned shard, dashed = gathered.

14. All-Gather: temporarily rebuild the current FSDP unit

(a) Before. Each rank persists one disjoint shard W_i of this unit's parameter tensor.

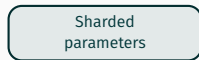


(b) After. Every rank materializes $W = [W_0 \parallel W_1 \parallel W_2 \parallel W_3]$; solid = owned shard, dashed = gathered.

All-Gather: **shards** $\rightarrow$ **full parameters**, but only for the unit currently being computed.

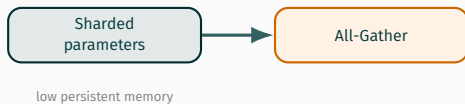
15. Forward pass: gather, compute, reshard

Sharded
parameters

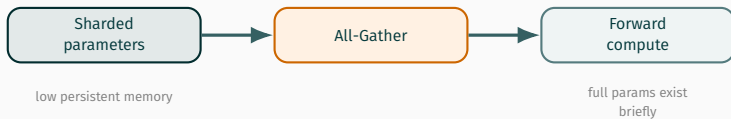


low persistent memory

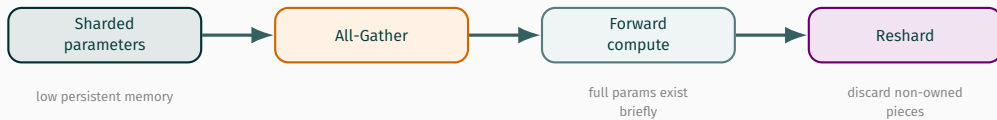
15. Forward pass: gather, compute, reshard



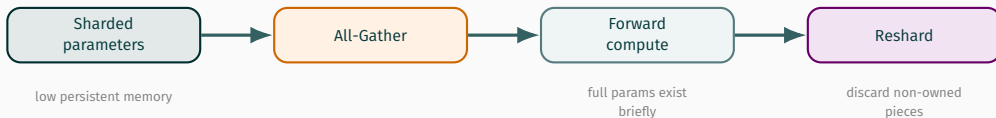
15. Forward pass: gather, compute, reshard



15. Forward pass: gather, compute, reshard



15. Forward pass: gather, compute, reshard



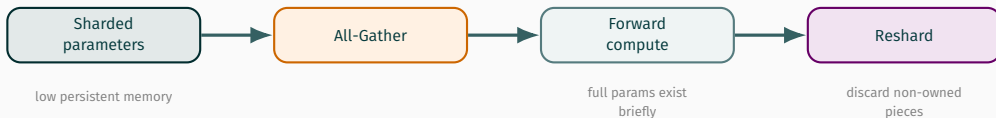
Resharding

After the forward computation, each GPU frees the full parameter and keeps only the shard it owns.

Why it saves memory

The expensive full representation is temporary; the sharded representation is the persistent one.

15. Forward pass: gather, compute, reshard



Resharding

After the forward computation, each GPU frees the full parameter and keeps only the shard it owns.

Why it saves memory

The expensive full representation is temporary; the sharded representation is the persistent one.

Forward = **All-Gather** → **compute** → **reshard**.

16. Why gathering does not destroy the memory benefit

Layer-wise FSDP units

Transformer block 1

Transformer block 2

Transformer block 3

Transformer block 4

Other blocks can stay sharded.

16. Why gathering does not destroy the memory benefit

Layer-wise FSDP units

Block 1: full

Transformer block 2

Transformer block 3

Transformer block 4

Other blocks can stay sharded.

Important distinction

FSDP gathers only the **current FSDP unit** (for example, one Transformer block), not the entire model.

16. Why gathering does not destroy the memory benefit

Layer-wise FSDP units

Transformer block 1

Block 2: full

Transformer block 3

Transformer block 4

Other blocks can stay sharded.

Important distinction

FSDP gathers only the **current FSDP unit** (for example, one Transformer block), not the entire model.

Peak memory is not simply $(P + G + O)/N$. Activations and temporary full-parameter buffers also occupy memory.

16. Why gathering does not destroy the memory benefit

Layer-wise FSDP units

Transformer block 1

Transformer block 2

Block 3: full

Transformer block 4

Other blocks can stay sharded.

Important distinction

FSDP gathers only the **current FSDP unit** (for example, one Transformer block), not the entire model.

Peak memory is not simply $(P + G + O)/N$. Activations and temporary full-parameter buffers also occupy memory.

Wrapping / grouping decides how large each FSDP unit is. Smaller units reduce the amount temporarily materialized at once.

16. Why gathering does not destroy the memory benefit

Layer-wise FSDP units

Transformer block 1

Transformer block 2

Block 3: full

Transformer block 4

Other blocks can stay sharded.

Important distinction

FSDP gathers only the **current FSDP unit** (for example, one Transformer block), not the entire model.

Peak memory is not simply $(P + G + O)/N$. Activations and temporary full-parameter buffers also occupy memory.

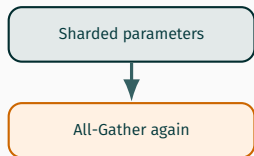
Wrapping / grouping decides how large each FSDP unit is. Smaller units reduce the amount temporarily materialized at once.

FSDP controls peak memory by gathering **one unit at a time**.

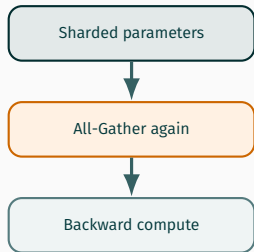
17. Backward pass: parameters are needed again

Sharded parameters

17. Backward pass: parameters are needed again



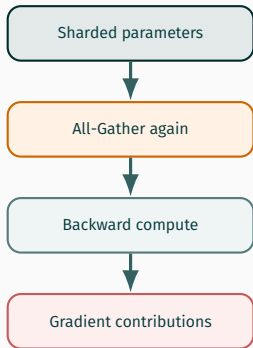
17. Backward pass: parameters are needed again



Why gather again?

Backpropagation needs the current layer's parameters to compute gradients with respect to inputs and weights.

17. Backward pass: parameters are needed again



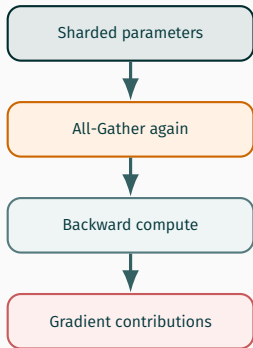
Why gather again?

Backpropagation needs the current layer's parameters to compute gradients with respect to inputs and weights.

New problem

Each GPU processed different mini-batch data, so their gradient contributions must be synchronized before the optimizer update.

17. Backward pass: parameters are needed again



Why gather again?

Backpropagation needs the current layer's parameters to compute gradients with respect to inputs and weights.

New problem

Each GPU processed different mini-batch data, so their gradient contributions must be synchronized before the optimizer update.

Backward creates distributed gradient contributions; FSDP must combine them efficiently.

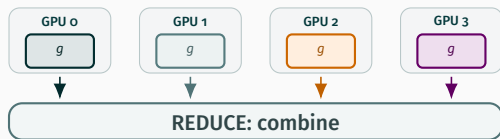
18. Reduce-Scatter: combine gradients, then split again

1. Gradient contributions

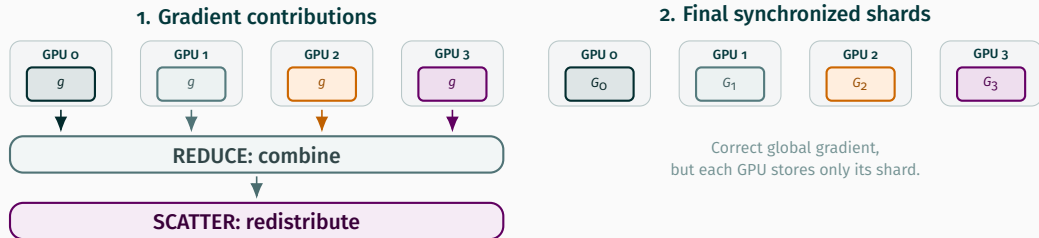


18. Reduce-Scatter: combine gradients, then split again

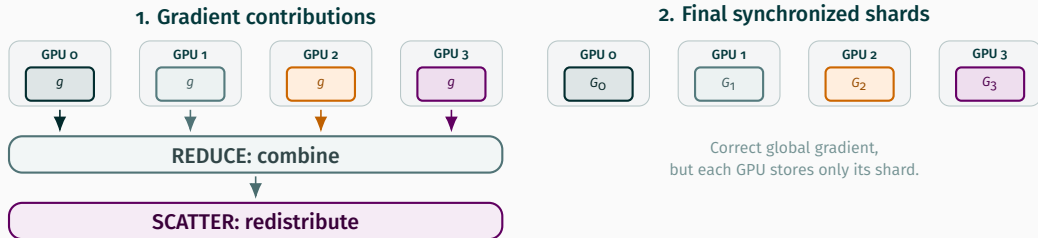
1. Gradient contributions



18. Reduce-Scatter: combine gradients, then split again



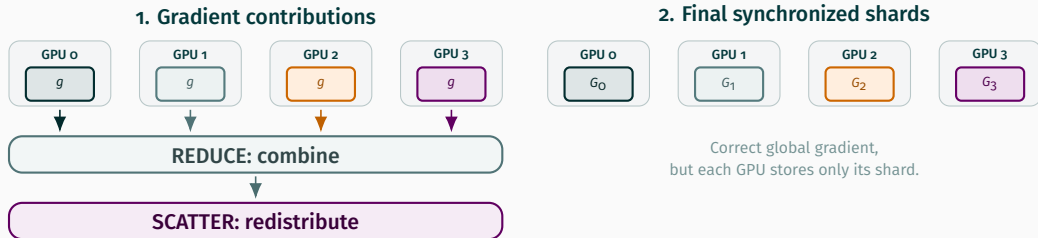
18. Reduce-Scatter: combine gradients, then split again



Compare with DDP

DDP typically uses **All-Reduce**: every rank ends with the full synchronized gradient. FSDP uses **Reduce-Scatter**: each rank ends with only its shard.

18. Reduce-Scatter: combine gradients, then split again

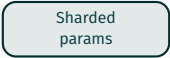


Compare with DDP

DDP typically uses **All-Reduce**: every rank ends with the full synchronized gradient. FSDP uses **Reduce-Scatter**: each rank ends with only its shard.

Reduce-Scatter = **combine** gradient contributions, then **redistribute** only the owned shard.

19. The complete FSDP training cycle

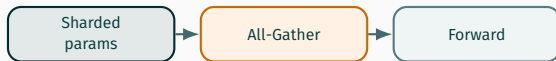


Sharded
params

19. The complete FSDP training cycle



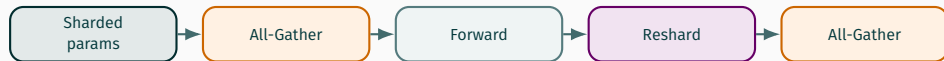
19. The complete FSDP training cycle



19. The complete FSDP training cycle



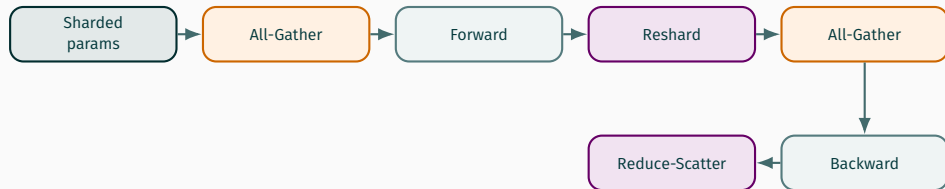
19. The complete FSDP training cycle



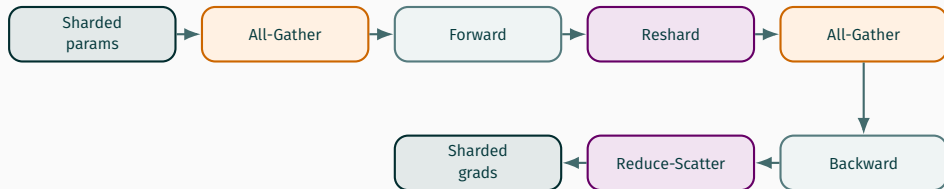
19. The complete FSDP training cycle



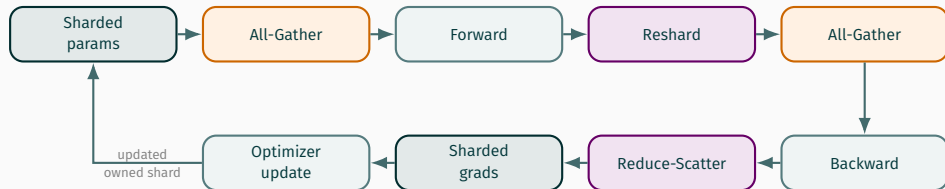
19. The complete FSDP training cycle



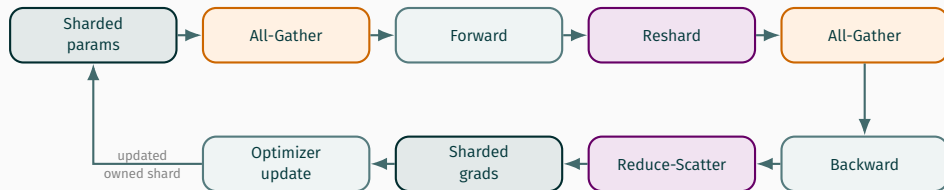
19. The complete FSDP training cycle



19. The complete FSDP training cycle

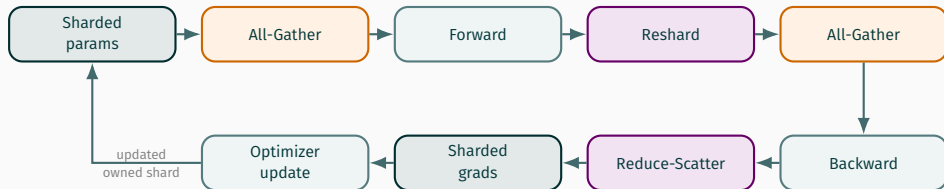


19. The complete FSDP training cycle



At the optimizer step, each GPU already owns the matching **parameter shard + gradient shard + optimizer-state shard**, so it can update that shard locally.

19. The complete FSDP training cycle



At the optimizer step, each GPU already owns the matching **parameter shard + gradient shard + optimizer-state shard**, so it can update that shard locally.

Keep model state divided for memory; gather parameters briefly only when computation needs them.

20. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

20. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

20. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

Overlap hides some cost

Compute block i

Gather block $i + 1$

Communication for the next unit can overlap with current computation.

20. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

Overlap hides some cost

Compute block i

Gather block $i + 1$

Communication for the next unit can overlap with current computation.

One-sentence summary

Shard most of the time; gather only when needed.

20. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

Overlap hides some cost

Compute block i

Gather block $i + 1$

Communication for the next unit can overlap with current computation.

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

One-sentence summary

Shard most of the time; gather only when needed.

FSDP enables larger-model training by trading **memory** for **communication**.